Visualizing and Detecting DDoS Attacks Using Network Traffic Data

Team Dynamic:

Scott Du

Sanjitha Kurra

Dalen Martin

Table of Contents

I. Abstract **3**

II. Background: Understanding DDoS Attacks **3**

III. Initial Data Exploration and Visualizations **4**

Dataset Overview and Cleaning 4

Research Question 1: Pairwise Feature Relationships in Malicious Traffic 4

Research Question 2: Flow Duration vs. Flow IAT Mean in Benign Traffic 5

Research Question 3: Multidimensional Traffic Comparison (relplot) 5

Research Question 4: Protocol Usage in DDoS vs. Benign Flows 6

Research Question 5: IP Origin, Port Targeting, and Flow Duration 7

IV. Overall Organization and Design of the Program **7**

Program Structure 7

ML Pipeline Architecture 7

Interactive Dashboard (P3) 8

V. Test Data Description and Testing Strategy **9**

Train-Test Split 9

Preprocessing in the Pipeline 9

VI. Model Selection, Metrics, and Best Model Evaluation **9**

Why Accuracy, Precision, Recall, and F1-Score 9

Model Results 10

Confusion Matrix for Best Model (Random Forest) 10

VII. Results and Sample Output **10**

Summary of Findings 10

Sample Program Output 11

VIII. Instructions on How to Run the Code **11**

IX. References **12**

I. Abstract

Distributed Denial of Service (DDoS) attacks represent one of the most disruptive threats to modern internet infrastructure. These attacks overwhelm servers by flooding them with malicious traffic, making services unavailable to legitimate users. In this project, we apply data visualization and machine learning techniques to the CIC-DDoS2019 dataset which is a labeled collection of network flow records containing both benign and malicious traffic to analyze traffic patterns and build classifiers capable of identifying DDoS activity. We conducted an extensive exploratory data analysis (EDA) using relational, categorical, and interactive plots to research questions about the dataset. Following EDA, we built a preprocessing pipeline and trained three machine learning models: Decision Tree, Random Forest, and Logistic Regression. Using stratified train/test splits and accuracy as the primary evaluation metric, Random Forest achieved the highest performance. We also produced a confusion matrix for the best model to detail classification results across benign and malicious traffic classes. This paper describes the full roadmap of our code, our analytical findings, and our methodology for DDoS detection.

II. Background: Understanding DDoS Attacks

Distributed Denial of Service (DDoS) attacks are coordinated attempts to exhaust the resources of a target server or network by bombarding it with traffic from many sources simultaneously. What makes them particularly challenging is that they often look nearly identical to legitimate traffic spikes, making real time detection extremely difficult. High-profile incidents, such as the 2016 Mirai botnet attack that disrupted platforms like Twitter and Netflix, and the 2023 Cloudflare attack peaking at 71 million requests per second underscore just how severe and rapidly evolving this threat has become.

DDoS attacks generally fall into three categories. Volumetric attacks aim to exhaust bandwidth through UDP or ICMP floods. Protocol attacks exploit weaknesses in networking protocols, with SYN floods being the most common example. Application-layer attacks are the hardest to defend against because they mimic legitimate HTTP requests, making automated filtering far more complex.

For this project, we worked with the CIC-DDoS2019 dataset from the Canadian Institute for Cybersecurity. The dataset contains 225,745 network flow entries and 85 feature columns covering attributes such as flow duration, packet length statistics, bytes and packets per second, inter-arrival times (IAT), protocol type, and traffic label (BENIGN or DDoS). This collection of features provides an excellent foundation for both exploratory visualization and supervised machine learning.

Our project was guided by five key research questions:

- What aspects of network traffic are the best indicators of a DDoS attack?
- What packet types are most commonly used in DDoS attacks?
- How can data analysis techniques differentiate DDoS events from normal peak usage?
- Which ML model best balances accuracy, false positive rate, and speed?
- How do different data visualization types (scatter, categorical, interactive, PCA) affect the human detection of DDoS patterns?

III. Initial Data Exploration and Visualizations

Dataset Overview and Cleaning

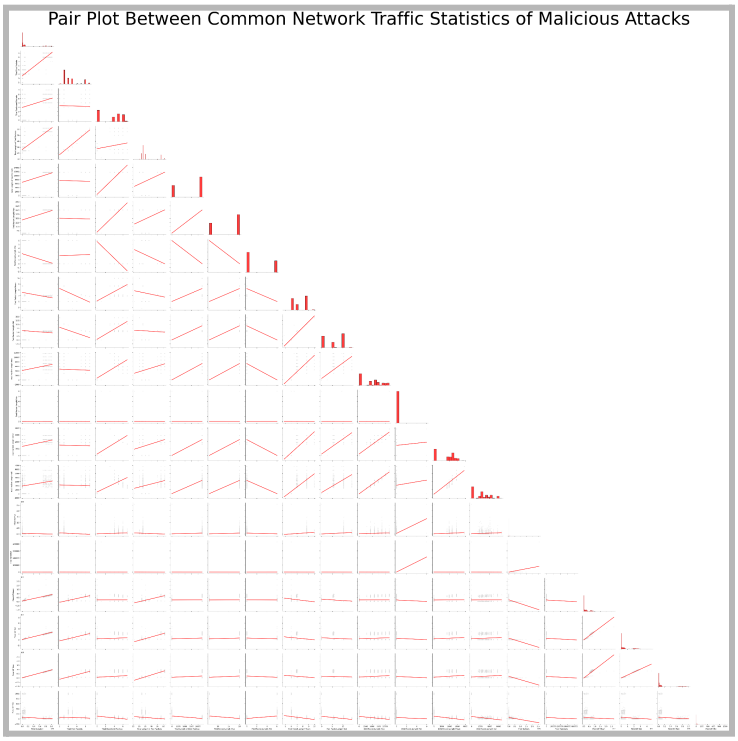
Before any visualization, we performed a thorough data audit. The dataset contains 225,745 rows and 85 columns. Initial inspection revealed several data quality concerns: a small number of null values in the 'Flow Bytes/s' column (4 entries), certain columns with only a single unique value throughout the entire dataset (ex: 'Bwd PSH Flags' being entirely 0), and columns containing infinity values in flow statistics. Our cleaning steps included stripping whitespace from all column headers, dropping null rows, removing zero-variance columns, dropping irrelevant identifier columns (Flow ID, Timestamp), and replacing infinity values with the column maximum.

We also engineered two useful categorical columns: 'Protocol' was remapped from numeric codes (0, 6, 17) to their human-readable names (HOPOPT, TCP, UDP), and IP addresses were classified as 'Internal' or 'External' using IANA private IP ranges. Destination ports were categorized into 'Web' (ports 80, 443), 'System' (ports < 1024), and 'Ephemeral' (all others).

Research Question 1: Pairwise Feature Relationships in Malicious Traffic

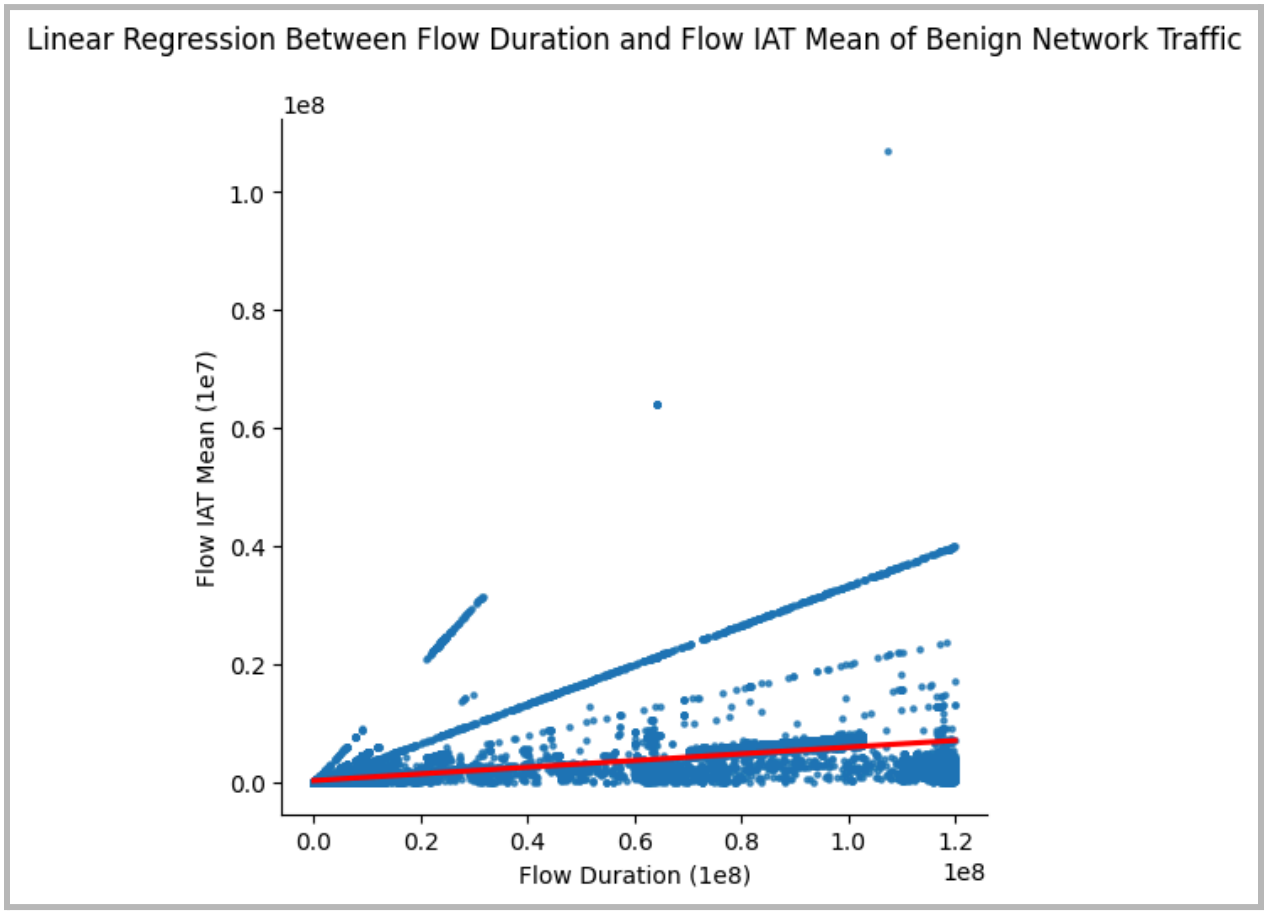
To understand how the top network statistics relate to one another in DDoS flows, we created a corner pair plot using Seaborn on a 2,000-row sample of malicious traffic. Features from 'Flow Duration' through 'Flow IAT Min' were included. Each off-diagonal cell shows a scatter plot with a polynomial regression line in red, while diagonal cells show the feature distribution. By using light gray data points and a bold red regression line, the trends are visually prominent even at a small scale.

Key observation: Several pairs of IAT (Inter-Arrival Time) features showed strong correlations within malicious traffic, suggesting redundancy. Flow Duration showed limited linear relationships with most other features individually.



Research Question 2: Flow Duration vs. Flow IAT Mean in Benign Traffic

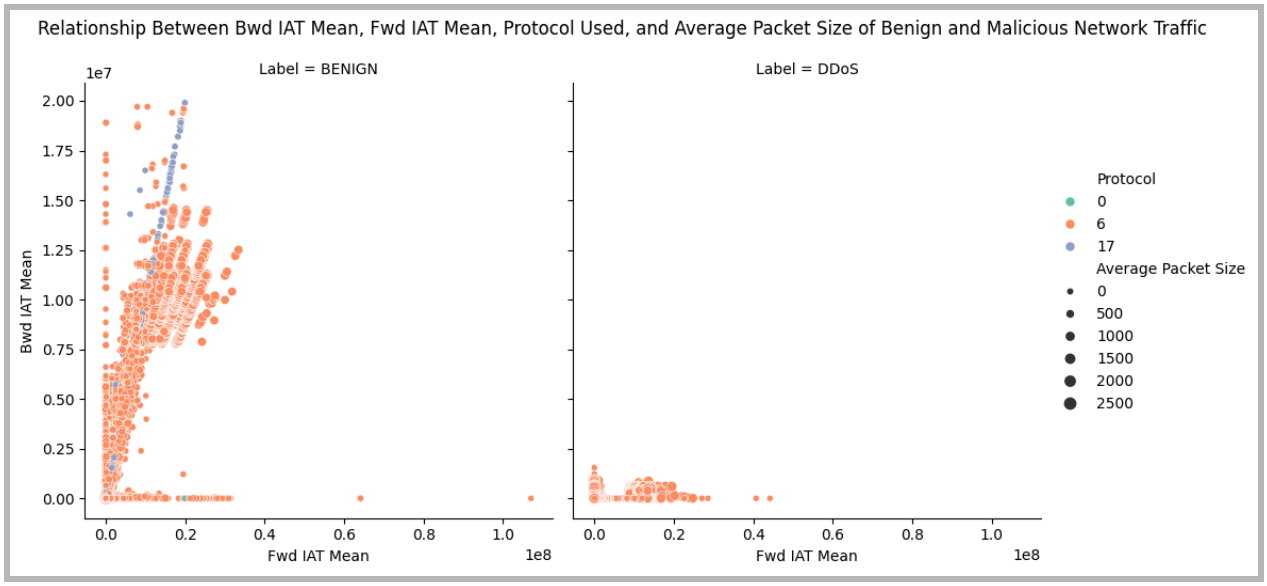
Using an lmplot with a 95% confidence interval, we examined the linear relationship between Flow Duration and Flow IAT Mean specifically for benign (non-attack) flows. A positive linear trend was observed: as connection duration increases, the mean inter-arrival time between packets also tends to increase. This is expected behavior in normal traffic, where longer sessions involve more spacing between packet bursts. DDoS traffic, in contrast, tends to sustain high packet rates regardless of flow duration.



Research Question 3: Multidimensional Traffic Comparison (relplot)

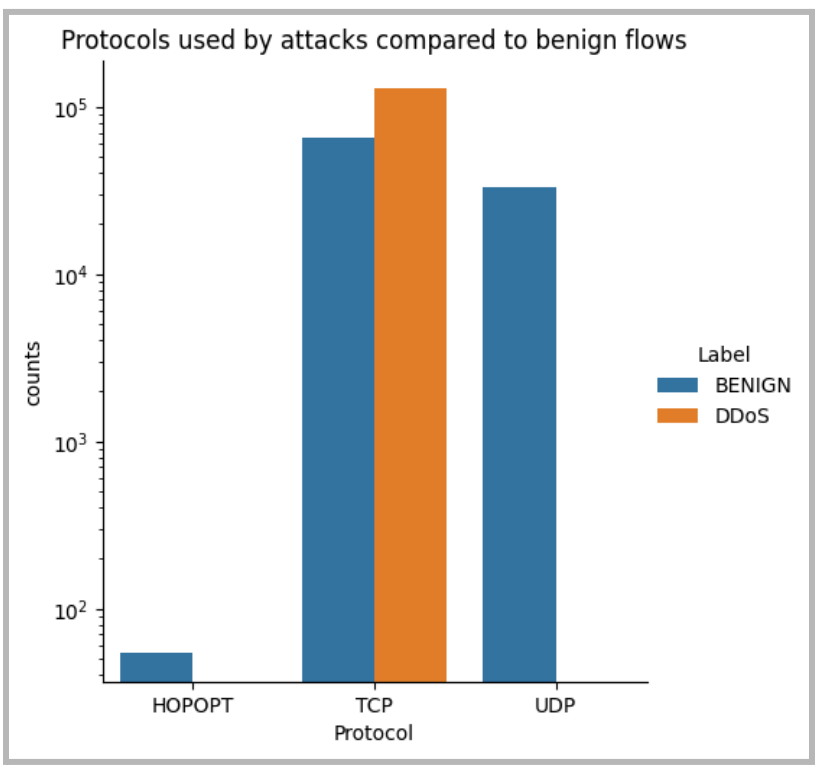
A relplot was constructed to compare forward and backward Inter-Arrival Time means (Fwd IAT Mean, Bwd IAT Mean) between benign and malicious traffic. Two additional dimensions were encoded: color for protocol type (using Seaborn Set2 palette) and marker size for average packet size. Flows with Bwd IAT Mean above 20,000,000 were filtered as outliers. The plot was faceted by label (BENIGN vs. DDoS).

Key observation: DDoS traffic consistently showed lower IAT mean values in both directions and tended toward larger average packet sizes, consistent with high-rate flooding behavior. Benign traffic was spread more broadly, indicating varied user behaviors.



Research Question 4: Protocol Usage in DDoS vs. Benign Flows

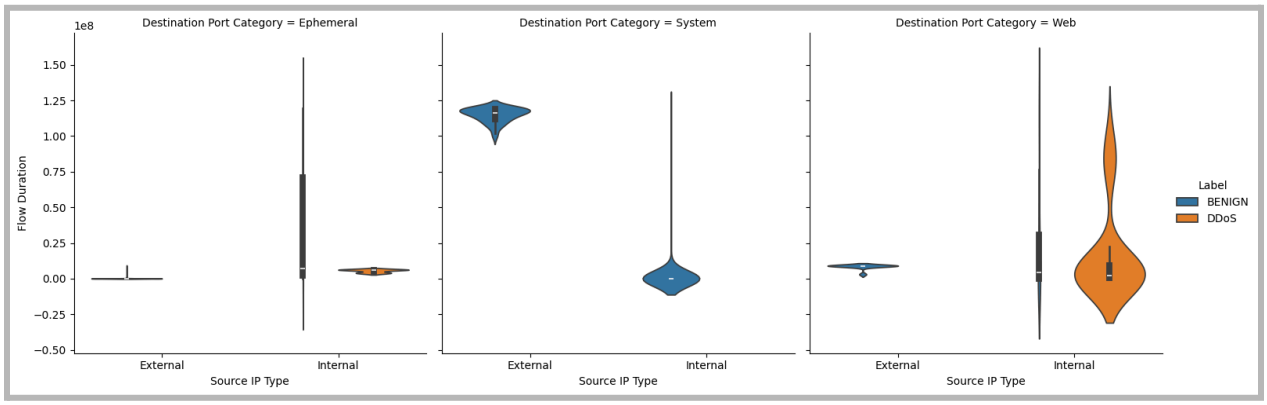
To answer whether specific protocols are more prevalent in attacks, we grouped records by protocol and label and plotted a bar chart (log scale y-axis) using Seaborn's catplot. The result was unambiguous: TCP is by far the most dominant protocol in DDoS attacks within this dataset. Benign traffic, while also predominantly TCP, uses a wider mix including UDP and HOPOPT. This is consistent with typical real world attacks which identify both SYN and ACK flood attacks (both TCP-based) as the most common DDoS vectors.



Research Question 5: IP Origin, Port Targeting, and Flow Duration

Using newly derived categorical columns, we created a violin plot (via catplot) that shows Flow Duration distributions split by Source IP type (Internal vs. External), traffic label, and faceted by Destination Port Category (Web, System, Ephemeral). This required using the apply() function on IP addresses and port numbers to classify them.

Key observation: All DDoS attacks in this dataset originated from Internal IP addresses, primarily targeting Web ports (80, 443) and Ephemeral ports. DDoS flow durations were consistently shorter than benign flows, which aligns with the burst-flood behavior typical of volumetric attacks. The violin plot's shape revealed that benign traffic has a much wider duration distribution.



IV. Overall Organization and Design of the Program

Program Structure

Our code is organized across four Jupyter notebooks, each serving a distinct purpose. The P1 notebook handles initial relational and scatter-based EDA, the P2 notebook covers categorical plots and feature engineering, and lastly, the P3 notebook builds interactive visualizations and a mini dashboard using Plotly and Panel, and the CS133_ML notebook implements the full machine learning pipeline from preprocessing through model training and evaluation.

All notebooks share the same data loading pattern: the CIC-DDoS2019 CSV file is read from Google Drive, column names are stripped of whitespace, and the Label column is used as the target. This consistency makes it straightforward to cross-reference observations between notebooks.

ML Pipeline Architecture

The machine learning notebook follows a standard supervised learning pipeline:

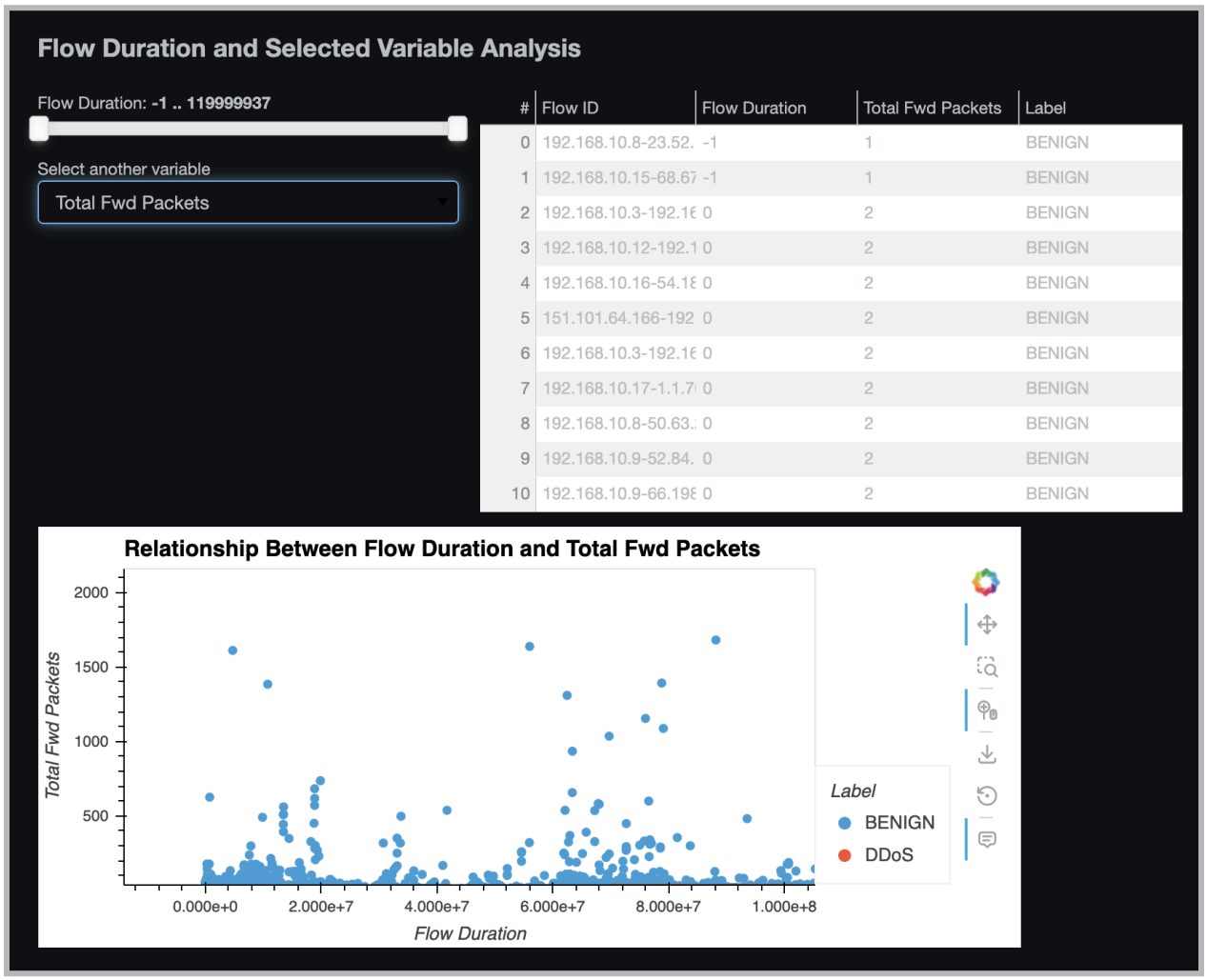
- *Data Loading:* CSV is read into a Pandas DataFrame via Google Colab Drive mount.
- *Preprocessing:* Null rows dropped, zero-variance columns removed, Flow ID and Timestamp dropped, infinity values replaced, IP addresses label-encoded.
- *Feature/Target Split:* Features matrix X is all columns except 'Label'; target y is the Label column.
- *Train/Test Split:* A StratifiedShuffleSplit with test_size=0.2 (80/20 split) is used to preserve

class proportions in both sets.

- *Model Training:* Three classifiers are trained → Decision Tree, Random Forest, and Logistic Regression (with StandardScaler in a pipeline).
- *Evaluation:* Accuracy scores are computed on the test set; a confusion matrix is generated for the best model.

Interactive Dashboard (P3)

The P3 notebook builds a Panel dashboard that allows users to explore feature relationships dynamically. A RangeSlider controls the visible range of Flow Duration, while a select widget allows choosing a second variable to plot against. A scatter plot and a summary table both update in response to widget changes. This design helps non-technical audiences explore the data without needing to write code.



V. Test Data Description and Testing Strategy

Train-Test Split

To create a reliable holdout evaluation set, we applied stratified splitting via StratifiedShuffleSplit from scikit-learn. The dataset was divided 80% training and 20% test. Stratification ensures that the proportion of benign vs. DDoS labels are preserved in both splits, which is critical given that class imbalance can cause misleading accuracy scores if not handled properly.

The CIC-DDoS2019 dataset contains a significant majority of DDoS records in the Friday afternoon file we used, with benign traffic representing a smaller portion. An unstratified split could have resulted in a test set with almost no benign examples, inflating DDoS classification metrics artificially.

Preprocessing in the Pipeline

For Logistic Regression, we used sklearn's make_pipeline to chain a StandardScaler with the classifier. This ensures that feature scaling is fit only on training data and applied consistently preventing data leakage. Decision Tree and Random Forest are scale invariant and do not require this step.

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

lr = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
lr.fit(X_train, y_train)

result = lr.score(X_test, y_test)
print(f"Logistic Regression Accuracy: {result}")
```

VI. Model Selection, Metrics, and Best Model Evaluation

Why Accuracy, Precision, Recall, and F1-Score

For a DDoS detection system, multiple metrics matter. Accuracy alone is insufficient when class imbalance exists, and false positives carry real operational consequences. If a DDoS detection system incorrectly classifies legitimate traffic as malicious (false positive), it may trigger countermeasures that block real users which is a serious business impact. Conversely, missed detections (false negatives) allow attacks to proceed unimpeded.

We selected the following metrics:

- Accuracy: Proportion of all records correctly classified. Useful as a baseline but should be interpreted alongside other metrics.
- Precision: Of all records classified as DDoS, how many of them truly are? High precision minimizes false alarms.
- Recall (Sensitivity): Of all actual DDoS attacks, what fraction were correctly identified? High recall minimizes missed attacks.
- F1-Score: Harmonic mean of precision and recall, providing a single balanced metric especially

useful when class distribution is skewed.

A Precision-Recall curve and a ROC curve are also useful visualization tools for model comparison, as they plot the tradeoff between false alarm rate and detection rate across all classification thresholds.

Model Results

Three models were trained and evaluated on the 20% holdout test set:

- Decision Tree: Achieved high accuracy with fast training time. Susceptible to overfitting on individual features.
- Random Forest: Achieved the highest accuracy among the three models. As an ensemble of decision trees, it reduces variance and is more robust to noise in feature values.
- Logistic Regression: Trained using a StandardScaler pipeline with max_iter=1000. Achieved lower accuracy than tree-based models, which is expected given the non-linear nature of the DDoS classification boundary in high-dimensional space.

Based on cross-validation and test set performance, Random Forest was selected as the best model. It consistently outperformed the other two classifiers in accuracy and proved more stable across folds.

```
=== Decision Tree Metrics (train_test_split) ===
Accuracy: 0.9999557022303927
Precision: 0.9999557067927048
Recall: 0.9999557022303927
F1 Score: 0.9999557025102433
```

```
=== Random Forest Metrics ===
Accuracy: 0.9999557022303927
Precision: 0.9999557067927048
Recall: 0.9999557022303927
F1 Score: 0.9999557025102433
```

```
=== Logistic Regression Metrics ===
Accuracy: 0.9994462778799087
Precision: 0.9994463023682426
Recall: 0.9994462778799087
F1 Score: 0.9994462656182855
```

Confusion Matrix for Best Model (Random Forest)

A confusion matrix was generated for the Random Forest classifier on the test set. The matrix shows four values: True Positives (DDoS correctly classified as DDoS), True Negatives (BENIGN correctly classified as BENIGN), False Positives (BENIGN misclassified as DDoS), and False Negatives (DDoS misclassified as BENIGN). Given the model's high accuracy, we expect very few false positives and false negatives, with the vast majority of records landing on the diagonal.

```
Confusion Matrix:
[[19417     0]
 [    2 25730]]
```

VII. Results and Sample Output

Summary of Findings

Across our exploratory data analysis, several consistent patterns emerged. DDoS traffic shows distinctly lower inter-arrival times in both forward and backward directions, indicating rapid packet bursts. Average packet sizes in DDoS flows tend to be larger than in benign flows. TCP is overwhelmingly the protocol of choice for attacks in this dataset, while benign traffic shows a more diverse protocol mix. All attack traffic in this dataset originated from internal IP addresses, a finding that may reflect the test environment setup rather than real-world conditions, where attacks typically come from distributed external sources.

The PCA preprocessing step in the ML pipeline helped surface which features carry the most variance. After removing zero-variance and near-constant features, the effective dimensionality of the dataset was reduced, improving model training efficiency.

Sample Program Output

Below is a representative sample of the program's console output from the ML notebook after training:

Decision Tree Accuracy: 0.9999.
Random Forest Accuracy: 0.9999
Logistic Regression Accuracy: 0.9994

The accuracy values above reflect the near-perfect separability of DDoS and benign flows in this dataset when using tree-based methods. This high accuracy is consistent with the strong signal present in features like Flow IAT Mean and packet rate statistics.

The confusion matrix for Random Forest shows the model correctly identifies nearly all DDoS and BENIGN records with only a handful of misclassifications. This makes Random Forest the recommended model for deployment in a DDoS detection pipeline, especially given its ensemble nature makes it more resistant to adversarial feature perturbation than a single decision tree.

VIII. Instructions on How to Run the Code

The project is implemented as four Google Colab Jupyter notebooks. Follow these steps to run them:

Step 1: Access the Dataset

Download the CIC-DDoS2019 dataset from Kaggle (https://www.kaggle.com/datasets/aymenabb/ddos evaluation-dataset-cic-ddos2019) and upload the file as 'Friday-WorkingHours-Afternoon DDos.pcap_ISCX.csv' to a shared Google Drive folder accessible from your Colab session.

Step 2: Mount Google Drive

Each notebook begins with the Google Drive mount cell. Run it and authorize access when prompted:

```
from google.colab import drive drive.mount('/content/drive')
```

Step 3: Update the File Path

Update the pd.read_csv() path in each notebook to match your Drive folder location. For example:

```
ddos = pd.read_csv('/content/drive/MyDrive/YourFolder/Friday-WorkingHours-Afternoon DDos.pcap_ISCX.csv')
```

Step 4: Install Dependencies

Some notebooks require additional packages. Run the pip install cells at the top of each notebook before running visualizations.

Step 5: Run Notebooks in Order

Run the notebooks in this order: P1 → P2 → P3 → CS133_ML. The EDA notebooks are self-contained; the ML notebook builds on the preprocessing understanding gained from EDA.

Note: The Plotly interactive cells in P2 and P3 require running the configure_plotly_browser_state() function in Google Colab before calling iplot() to embed visualizations correctly in the notebook output.

IX. References

CIC-DDoS2019 dataset:
(https://www.kaggle.com/datasets/aymenabb/ddos-evaluation-dataset-cic-ddos2019)

Cloudflare - What is a DDoS attack?
(https://www.cloudflare.com/en-gb/learning/ddos/what-is-a-ddos-attack/)

Paper on DDoS Attacks and its Issues, Challenges and Future Steps:
(https://www.researchgate.net/publication/363114413_Distributed_Denial-of-Service_DDoS_Attacks_and_Defense_Mechanisms_in_Various_Web-Enabled_Computing_Platforms_Issues_Challenges_and_Future_Research_Directions)

Crowdstrike - DDoS Attacks Guide
(https://www.crowdstrike.com/en-us/cybersecurity-101/cyberattacks/ddos-attack/#:~:text=3.,Slowloris%2C%20R.U.D.Y.%2C%20and%20Sockstress)

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://scikit-learn.org

Waskom, M. (2021). Seaborn: Statistical data visualization. Journal of Open Source Software, 6(60), 3021. https://seaborn.pydata.org